



Audit Ledger

Your tamper-evident record of everything your AI agents did — and how to prove, to anyone, that it was not altered.

User Guide · Free edition

Version 1.0 · 2026-07-21 Audience: developers, ops, compliance & anyone who needs the receipts

This guide explains what the Audit Ledger is, why it matters, and exactly how to use it — in plain language first, with the technical detail alongside for those who want it. Every screenshot is the real product. If you only read one thing: the **Verify integrity** button runs a check *in your own browser* that proves your records are intact — you never have to take Tracelane's word for it.

1 · What the Audit Ledger is

The Audit Ledger is an **append-only, tamper-evident record** of what Tracelane's gateway did on your behalf: every AI-provider request it proxied and every guardrail or eval verdict it produced. Each entry is written once and never changed. Entries are cryptographically linked together, so if anyone alters, removes, or reorders even a single past entry, the change becomes **provable** — not just to Tracelane, but to any independent party.

THE ONE-SENTENCE VERSION

It is a sealed logbook for your agents: you can show an auditor exactly what ran, and mathematically demonstrate that the log was not edited after the fact.

What it is not: it is not your full trace data. Rich, full-fidelity spans (prompts, tool calls, timings) live on the **Traces** page. The Audit Ledger is the smaller, sealed *evidence* record — the part designed to stand up to scrutiny.

2 · Why it matters

If you are...	The Audit Ledger gives you...
A developer	A trustworthy history of what your agents actually did in production — no “trust me” logs that could have been quietly edited.
An ops / on-call engineer	An incident-grade record: when something goes wrong, the sequence of events is fixed and verifiable.
A compliance / risk officer	Evidence you can hand to an auditor or regulator (e.g. under the EU AI Act’s record-keeping expectations) that is independently verifiable without trusting the vendor.
A customer of yours	Assurance you can demonstrate what your AI did on their behalf, and prove the record is genuine.

3 · How it works (in plain terms)

Two mechanisms work together. You do not need to understand the math to trust the result — but here is what is happening.

a. The hash chain — tamper-evidence

Every entry gets a **fingerprint** (a SHA-256 hash) computed from its contents. Each entry also carries the fingerprint of the entry *before* it — like each page of a ledger citing the exact wording of the previous page. Change any past entry and its fingerprint changes, which breaks the link to the next entry, and to every entry after it. So a single silent edit is impossible to hide: re-computing the fingerprints exposes exactly where the break is.

WHO CARES, CONCRETELY?

An auditor asks “how do I know this log wasn’t cleaned up before you showed me?” The hash chain is the answer: they re-compute the fingerprints themselves and see that every link holds — or see precisely which entry was tampered with.

b. Public anchoring — independent, third-party proof

Your entries are grouped into **batches** (about 100 events each). For each batch, a single summary fingerprint (a *Merkle root*) is published to **Sigstore’s public transparency log** — an append-only, tamper-evident log run by the open-source community, independent of Tracelane. This is like getting a notary’s timestamp on each batch. Because the proof lives in a public log Tracelane does not control, it demonstrates your records existed, in that exact form, at that time — even if you stopped trusting Tracelane entirely.

Anchoring runs from your ledger’s very first entry (its *genesis*) onward. One anchor is published per batch, so the number of anchors grows as your ledger grows.

4 · The page, section by section

Open **Audit** from the top navigation. Here is what you see and what each part means.

The trust panel & “Verify integrity”

This is the dominant panel at the top. Before you verify, it shows a neutral **Ready to verify** state explaining what the check does. The single orange **Verify integrity** button runs the check.

◆ Ready to verify

What this does: re-hashes every event and re-checks each link to the one before it, starting from your chain's genesis — proving the events are intact and in order (any change would break a hash). It runs entirely **in your browser** over the ledger (SHA-256, domain-separated) — nothing is trusted from our server, so a green result is one you reproduced yourself.

Verify integrity

◆ Publicly anchored (Sigstore Rekord v2) 1 batch anchored

ANCHORED ROOTS IN log2025-1.rekor.sigstore.dev · checkpoint ↗

Log Index 18812240

Clicking that link opens the log's signed checkpoint — raw text showing the log origin, tree size, current root hash, and the log's own signature over them. This is expected output, not an error; it is the one independently-fetchable public artifact. You verify each anchored batch root against it offline using the inclusion proof bundled in your downloaded evidence.

Rekor v2 is a tiled log with no per-entry web page. Each root's inclusion proof + the log's signed checkpoint travel in your downloaded evidence and verify offline against the pinned log key — confirm the live log with curl https://log2025-1.rekor.sigstore.dev/checkpoint.

Events 4 Chain head d92101f3969e... Audit signing key +n4+cpXk9SZH8BK/... verify this key ↗

The trust panel in its “Ready to verify” state — the check has not run yet.

“About this ledger” — scope, anchoring & retention

A supporting panel that states what the ledger covers, its **Public anchoring** count (batches anchored within the events shown), and its **Retention** — the ledger is append-only and never expires, unlike your plan's trace-retention window.

About this ledger

An **append-only, tamper-evident** record of what the gateway did — every proxied request and every guardrail / eval verdict — so you can prove to an auditor exactly what ran and that the record was not altered. It covers **gateway-proxied traffic**; full-fidelity spans sent via the SDK / OTLP live in Traces and are not part of this chain.

PUBLIC ANCHORING ⓘ

1 anchored

batches within the events shown · Sigstore Rekord v2

RETENTION ⓘ

Append-only

no automatic expiry

► VOLUME DETAIL

EVENT TYPES RECORDED (THIS WINDOW)

chat.completions.request 2 guardrail.verdict 1 eval.verdict 1

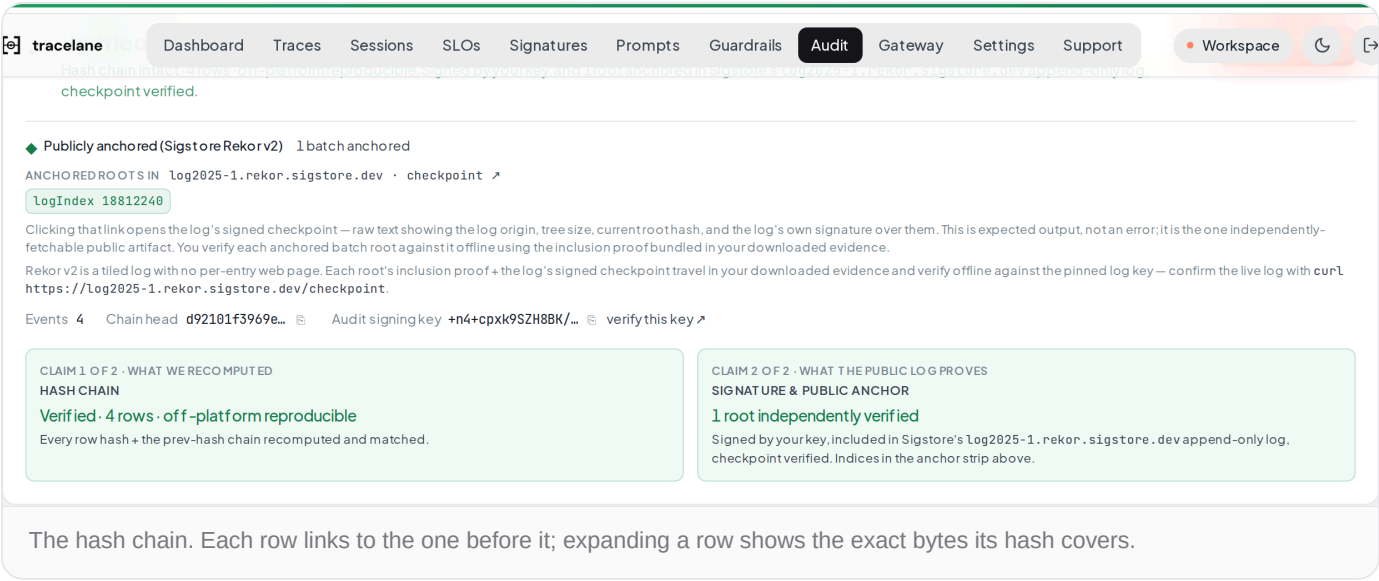
Public anchoring and retention. The anchoring count reflects the batches within the events loaded on screen; the complete set travels in the export.

WHY THE ANCHORING NUMBER CAN LOOK “SMALL”

The page loads a fast, verifiable preview (the first ~1,000 events from genesis). If your ledger has, say, 2,500 events across 25 batches, the preview contains the first 10 batches — so you'll see “10 anchored”. That is not a cap on your ledger; the complete ledger (all 25 batches) is in the export. The preview is sized so the in-browser check stays instant even for very large ledgers.

The hash chain

A visual walk of the entries, newest linkage highlighted, ending in the **chain head** (the fingerprint of the latest loaded entry). Expand any row to see exactly the data its fingerprint covers — full transparency into what is being hashed.



5 · Walkthrough: verify integrity

- 1 **Open the Audit page.** The trust panel shows “Ready to verify”.
- 2 **Click “Verify integrity”.** The verifier — open-source code — runs entirely in your browser. Nothing is sent to Tracelane; nothing is taken on trust.
- 3 **Read the verdict.** Within a moment you get a plain-English result and a two-claim breakdown (below). A green result is one *you* reproduced.

WHY “IN YOUR BROWSER” MATTERS

The check does not ask Tracelane “is this valid?” and trust the answer. It re-does the math locally over data you can inspect. That is what makes it credible evidence rather than a vendor claim.

6 · Reading the result

The verdict is deliberately split into **two claims**, never one blurred “OK”, so you always know exactly what was proven.

Claim	Plain meaning
Claim 1 — what we recomputed (Hash chain)	The verifier re-computed every entry’s fingerprint and every link between entries, locally. If this is green, your records are intact and in order.
Claim 2 — what the public log proves (Signature & public anchor)	Your batch fingerprints were signed by your key and found in Sigstore’s public log, checkpoint verified — independent, third-party confirmation.

Outcome A — Verified green

The chain is intact, your key’s signature checks out, and at least one batch is confirmed in the public log. This is the strongest state.

✓ **Verified**

Hash chain intact · 4 rows · off-platform reproducible. Signed by your key, and 1 root anchored in Sigstore's log2025-1.rekor.sigstore.dev append-only log, checkpoint verified.

Verify integrity

◆ Publicly anchored (Sigstore Rekor v2) 1 batch anchored

ANCHORED ROOTS IN log2025-1.rekor.sigstore.dev · checkpoint ↗

LogIndex 18812240

Clicking that link opens the log's signed checkpoint — raw text showing the log origin, tree size, current root hash, and the log's own signature over them. This is expected output, not an error; it is the one independently-fetchable public artifact. You verify each anchored batch root against it offline using the inclusion proof bundled in your downloaded evidence.

Rekor v2 is a tiled log with no per-entry web page. Each root's inclusion proof + the log's signed checkpoint travel in your downloaded evidence and verify offline against the pinned log key — confirm the live log with `curl https://log2025-1.rekor.sigstore.dev/checkpoint`.

Events 4 Chain head d92101f3969e... Audit signing key +n4+cpXk9SZH8BK/... verify this key ↗

CLAIM 1 OF 2 · WHAT WE RECOMPUTED
HASH CHAIN
Verified · 4 rows · off-platform reproducible
Every row hash + the prev-hash chain recomputed and matched.

CLAIM 2 OF 2 · WHAT THE PUBLIC LOG PROVES
SIGNATURE & PUBLIC ANCHOR
1 root independently verified
Signed by your key, included in Sigstore's log2025-1.rekor.sigstore.dev append-only log, checkpoint verified. Indices in the anchor strip above.

“Verified”: hash chain intact, signed by your key, root anchored in Sigstore's public log (with its logIndex), checkpoint verified.

Outcome B — Chain verified green (qualified)

The chain is intact and signed, but no public anchor happened to fall inside the on-screen preview (e.g. your newest batch hasn't been anchored yet). Still tamper-evident — run the export + CLI for the public-log proofs across the whole ledger.

Outcome C — Integrity check failed red

Something does not add up: a fingerprint doesn't match, a link is broken, or an anchor proof is missing. The page tells you the first failing entry and — importantly — **what to do next**.

✗ **Integrity check failed**

The hash chain is broken at seq 0 — see the chain view below.

Verify integrity

What this means & what to do next

- This is the check working, not a product bug: the verifier ran in your browser and the ledger it saw does **not** match its own hashes. Something altered the events after they were recorded — most often the exported file was edited or truncated after download.
- Re-download a fresh copy from **Download the complete ledger** below and verify again. A clean export that still fails points at the stored ledger, not your file.
- If a fresh export still fails, treat it as a potential integrity incident and **contact Tracelane support** with the first failing seq shown below and your export attached. Nothing in the app can “fix” a broken chain — that is the point of tamper-evidence.

◆ Publicly anchored (Sigstore Rekor v2)

ANCHORED ROOTS IN log2025-1.rekor.sigstore.dev · checkpoint ↗

LogIndex 18812240

Clicking that link opens the log's signed checkpoint — raw text showing the log origin, tree size, current root hash, and the log's own signature over them. This is expected output, not an error; it is the one independently-fetchable public artifact. You verify each anchored batch root against it offline using the inclusion proof bundled in your downloaded evidence.

Rekor v2 is a tiled log with no per-entry web page. Each root's inclusion proof + the log's signed checkpoint travel in your downloaded evidence and verify offline against the pinned log key — confirm the live log with `curl https://log2025-1.rekor.sigstore.dev/checkpoint`.

Events 4 Chain head d92101f3969e... Audit signing key +n4+cpXk9SZH8BK/... verify this key ↗

CLAIM 1 OF 2 · WHAT WE RECOMPUTED
HASH CHAIN
Chain broken — recomputed hashes do not match
at seq 0: row_hash_mismatch

CLAIM 2 OF 2 · WHAT THE PUBLIC LOG PROVES
SIGNATURE & PUBLIC ANCHOR
Anchor in log, but rows changed
The anchored root is still in the public log, but the ledger's rows no longer match it — see the broken chain (Claim 1).

A failure: the red verdict names the first broken `seq`, and the “What this means & what to do next” block walks you through remediation. Both claims turn red with the specific reason.

7 · Scenario flows: if this, then that

Verified (green)	You're done. This is your evidence. For a formal hand-off, download the complete ledger (§8) and archive it — the download <i>is</i> the record an auditor can independently verify.
Chain verified (no anchor in view)	Normal for a very recent or small ledger. If you need public-log proof now, download the complete ledger and run <code>tlane verify</code> — it checks every batch, including ones outside the on-screen preview.
Failed — first time	Re-download a fresh copy from “Download the complete ledger” and verify again. Most failures are a downloaded file that was edited or truncated locally — a fresh export clears that.
Failed — fresh export still fails	Treat it as a potential integrity incident. Email support@tracelane.dev with the first failing <code>seq</code> (shown on the page) and the export attached. Nothing in the app can “fix” a broken chain by design — that is exactly what tamper-evidence is for.
No events yet	The ledger starts with your first gateway-proxied call. Point an agent at the Tracelane gateway and entries appear here.

8 · Getting the complete ledger

The on-screen chain is a fast preview. The **complete, uncapped ledger** — the file an auditor verifies — is the **export**. On the Audit SKU this is a one-click download; on other plans you can see and verify your preview for free, and the downloadable evidence pack is the paid add-on.

complete chain from genesis · 4 events

The export card. The downloaded file is independently-verifiable evidence you can hand to a regulator and check with the open-source CLI — no Tracelane account required.

FREE VS. PAID

Free on every plan: see your chain from genesis, run the in-browser “Verify integrity”, read both claims. **Audit SKU add-on:** download the complete, uncapped evidence pack and verify it offline with the `tlane verify` command-line tool — the format built for regulator hand-off. The *Compliance & Evidence Handbook* (available to Audit-SKU customers) covers that end-to-end.

9 · Glossary

Tamper-evident

Changes cannot be hidden. Altering a record doesn't become *impossible* — it becomes *detectable* by anyone who checks. (We say “tamper-evident”, never “tamper-proof”.)

Hash / fingerprint (SHA-256)

A short, fixed-length value computed from data. Any change to the data yields a completely different hash, so hashes act as fingerprints.

Hash chain

A sequence where each entry includes the previous entry's hash, linking them so a change anywhere breaks the chain from that point on.

Entry / `seq`

One record in the ledger. `seq` is its sequence number, starting at 0 (genesis).

Genesis

The very first entry (`seq 0`). Verification always starts here — it is why the on-screen view begins at genesis.

Chain head

The hash of the most recent entry in view — the current “tip” of the chain.

Batch & Merkle root

Entries are grouped into batches (~100 events). A Merkle root is one fingerprint that summarizes a whole batch, so a single value can vouch for many entries.

Anchor

Publishing a batch's Merkle root to a public log, creating independent proof it existed in that form at that time.

Sigstore / Rekor

An open-source, publicly-run transparency log. Tracelane anchors your batch roots here so the proof does not depend on trusting Tracelane.

logIndex

The position of an anchored entry within the public Rekor log — a coordinate that, with the log, locates the proof.

Checkpoint

The public log's own signed statement of its current size and root. Your anchors are verified against it, offline.

Signature (Ed25519)

Your ledger's batch roots are signed by a key unique to your workspace, so an anchor is bound to *you*.

Evidence pack

The complete exported ledger (NDJSON) plus the proofs needed to verify it independently.

10 · FAQ

Is “Verify integrity” just a demo?

No. It runs the real, open-source verifier in your browser over your real ledger. A green result is one you reproduced yourself, not a status Tracelane asserted.

Does verifying send my data anywhere?

No. The check is entirely local to your browser. Public-anchor proofs travel *with* your evidence, so even that check needs no live network call to Tracelane.

Why does the anchoring count seem stuck (e.g. “10”)?

The page verifies a preview of the first ~1,000 events. The anchor count reflects the batches inside that preview, not your whole ledger. Your complete set of anchors is in the export.

What decides batch size and how many batches I have?

Events are grouped into fixed-size batches of about 100 as they accrue; one anchor is published per batch. So the batch count grows one-per-~100-events, from genesis onward.

My ledger has millions of events — can the page show all of them?

The browser view is a verifiable preview by design (you can't hash millions of rows in a tab). The complete chain is verified out-of-band via the export and the `tlane verify` CLI, which streams the entire ledger with no cap.

What should I do if verification fails?

Re-download a fresh export and verify again; most failures are a locally-edited/truncated file. If a clean export still fails, email support@tracelane.dev with the first failing `seq` — it's handled as a potential integrity incident.

Can Tracelane secretly change my past records?

Not without it becoming detectable. That's the whole point: an alteration breaks the hash chain and diverges from what's already anchored in the public log.

Is the record in my local time?

All timestamps are shown in **UTC** and labeled as such, so records from different regions and viewers never disagree about when something happened.

11 · Getting help

Support: support@tracelane.dev — or use the in-product **Support** page (top navigation). We follow up by email.

For a verification failure, include the first failing `seq` shown on the Audit page and attach the exported ledger. That is everything we need to investigate.

Tracelane Audit Ledger — User Guide (Free edition) · Version 1.0 · 2026-07-21. Screenshots are of the live product. This guide describes how to use and verify the Audit Ledger; it does not disclose Tracelane's internal implementation.